

시스템 구조 정리

AI 기술문의 자동 답변 시스템

WebSquare 기술지원 문의가 들어왔을 때 기존 문의, 가이드, API 문서를 먼저 찾고, 첨부 내용까지 확인한 뒤 담당자가 검토할 답변 초안을 만드는 구조입니다.

REST 게시판/운영 UI 연동	RAG Vector + BM25 하이브리드 검색	OCR 첨부 이미지 로그 분석	검증 정책 판단 + API 존재 확인
-----------------------------	--------------------------------------	----------------------------	--------------------------------

작성 기준

현재 개발된 코드 기준

주요 파일

src/api, src/generator, src/rag, src/collectors

주요 런타임

Node.js Express, Python RAG Server, ChromaDB, Codex Exec, Tesseract OCR

산출물

시스템 아키텍처 PDF

1. 전체 시스템 구조

그림 1. 전체 흐름



구조 요약

API 서버는 요청을 받고 결과를 돌려주는 역할만 맡습니다. 답변을 만드는 과정은 `AnswerPipeline` 에 모았습니다. 검색은 Python RAG 서버로 분리했고, 수집/분류/인덱싱은 답변 요청과 따로 돌리는 배치 작업으로 구성했습니다.

문의 처리 경로

고객 문의 → Express API → Answer Pipeline → RAG/MCP/OCR/LLM → API 검증 → 답변/출처/검토상태 반환

데이터 갱신 경로

수집원 → raw 저장 → 통합/분류 → 임베딩 생성 → ChromaDB 저장 → 검색 서버에서 사용

2. 시스템 구성도

그림 2. 실행 구성 기준

구분	구성 요소	주요 역할	저장/연동 위치
사용자 영역	기술지원 게시판 운영/검토 화면 CLI 테스트	문의 입력, 답변 확인, 담당자 검토	답변 요청 검색 요청 테스트 실행
API 영역	Express API 서버 라우터 API Key 인증	요청 접수, 인증, 검색/답변/큐/첨부 기능 연결	API 서버 검색/답변/큐/첨부 인터페이스
업무 처리 영역	답변 파이프라인 답변 생성기 정책 판단 API 검증	질문 분류, 근거 검색, 답변 생성, 검증, 저장	답변 생성 엔진 정책 판단 검증 처리
검색 영역	Python RAG 서버 ChromaDB BM25 인덱스	유사 문의 검색, 키워드 검색, 검색 결과 점수화	검색 서버 벡터 저장소 키워드 인덱스
데이터 영역	수집 데이터 통합 Q&A 분류 데이터 답변 저장 파일	검색 근거 관리, 생성 답변 이력 보관	원본 데이터 정제 데이터 답변 이력
외부 연동	Gmail W-Tech Confluence API Guide Codex Exec	지식 수집, 공식 문서 확인, LLM 답변 생성	운영 설정 기준 연동

주요 구성 요소

구성 요소	담당 기능	입력	출력
API 서버	REST API, 인증, 요청 라우팅	질문, 첨부, 버전 정보	답변, 출처, 검토 상태
답변 생성 엔진	답변 생성 전체 흐름 제어	정리된 질문, RAG 결과, 첨부 컨텍스트	답변 초안, 검증 결과, 저장 파일
RAG 검색 서버	벡터/BM25 검색, 인덱싱	분류 완료 Q&A, 검색어	유사 문의, 검색 점수, 컨텍스트
데이터 수집 배치	외부 지식 데이터 수집	Gmail, W-Tech, Confluence, API Guide	원천 JSON, 통합 Q&A
분류기	문의 유형 분류	통합 Q&A 또는 신규 질문	카테고리, 서브카테고리
공통 처리	설정 로딩, 민감정보 마스킹, 파일 변환	설정값, 문자열, 첨부 정보	정리된 설정값, 마스킹 결과, 변환 결과

데이터 보관 구조

데이터 구분	내용	사용 시점
원본 데이터	Gmail, W-Tech, Confluence, API Guide에서 수집한 원본	수집 직후
통합 Q&A	수집 데이터를 하나의 문의/답변 형식으로 맞추는 데이터	분류 전
분류 데이터	문의 유형과 카테고리가 붙은 검색 대상 데이터	검색 인덱스 생성 전
검색 인덱스	벡터 검색과 키워드 검색에 사용하는 인덱스	문의 검색 시
답변 이력	생성된 답변 초안과 처리 결과	답변 생성 후
운영 설정	연동 계정, API Key, OCR, LLM, MCP 설정	서버/배치 실행 시

3. 답변 생성 처리 순서

그림 3. 문의 1건이 처리되는 순서

단계	화면/요청자	API 서버	답변 생성	검색/보조 기능	LLM/결과
1. 요청	질문, 버전, 첨부 전달	API Key 확인 요청값 정리			
2. 범위 확인		라우터에서 pipeline 호출	민감정보 마스킹 분류 답변 범위 확인		blocked면 안내문 즉시 반환
3. 자료 검색			질문 핵심어 추출 검색 결과 확인	RAG 서버 검색 실패 시 subprocess 대체 필요 시 보정 검색	
4. 내용 보강			RAG + MCP + 첨부 + 미확인 용어 병합	MCP 스펙 조회 Tesseract OCR API Guide 용어 확인	
5. 답변 생성			프롬프트 구성 답변 범위 포함		Codex Exec 호출 한국어 답변 보정
6. 확인/저장	답변 확인	JSON 응답 반환	API명 검증 필요 시 재생성 답변 파일 저장 검토 큐 등록	API Guide/RAG DB 확인	answer, sources, risk 반환

답변 처리 기준

모드	조건	처리	운영 의미
auto	일반 기술 문의이고 참고 근거가 있는 경우	답변 초안 생성 후 API명 확인	담당자 검토 후 활용
human_review	보안, 권한, 장애 영향 등 확인이 필요한 문의	답변은 만들지만 검토 필요로 표시	담당자 확인 후 처리
blocked	라이선스, 데모, 계약, 파일 제공 등 기술 답변 범위 외 요청	검색/LLM을 거치지 않고 안내 문 반환	자동 답변 대상에서 제외

확인하는 지점

입력 확인 요청값 정리, API Key, 민감정보 마 스킹	근거 확인 RAG 결과, 보정 검색, MCP 스펙 보 강	출력 확인 API명 확인, 재생성, 검토 큐 등록
---	--	---------------------------------------

4. RAG 및 데이터 구조

1. 수집 Gmail W-Tech Confluence API Guide	2. 원본 저장 <code>data/raw/*.json</code> 수집원별 저장	3. 통합 <code>all_qa.json</code> 중복/형식 정리	4. 분류 카테고리 서브카테고리	5. 색인 e5 embedding ChromaDB 저장	6. 검색 RAG server Vector + BM25
--	--	--	--------------------------------	---	---

수집 계층	<code>src/collectors/index.js</code> 에서 수집기를 실행합니다. 대상은 Gmail, W-Tech, Confluence, API Guide, W-Tech FAQ입니다. Puppeteer를 사용할 수 없는 서버에서는 W-Tech 계열 수집을 건너뛸 수 있습니다.
--------------	--

정제/분류 계층	수집 결과를 통합 Q&A로 합치고 문의 유형별 카테고리를 붙입니다. 이 결과가 검색 인덱스의 문서 본문과 메타데이터로 들어갑니다.
-----------------	--

검색 계층	<code>searcher.py</code> 는 벡터 검색과 BM25 검색을 같이 수행합니다. 최종 점수는 벡터 0.6, BM25 0.4 가 중치로 합산하고, 상위 결과만 답변 컨텍스트로 넘깁니다.
--------------	--

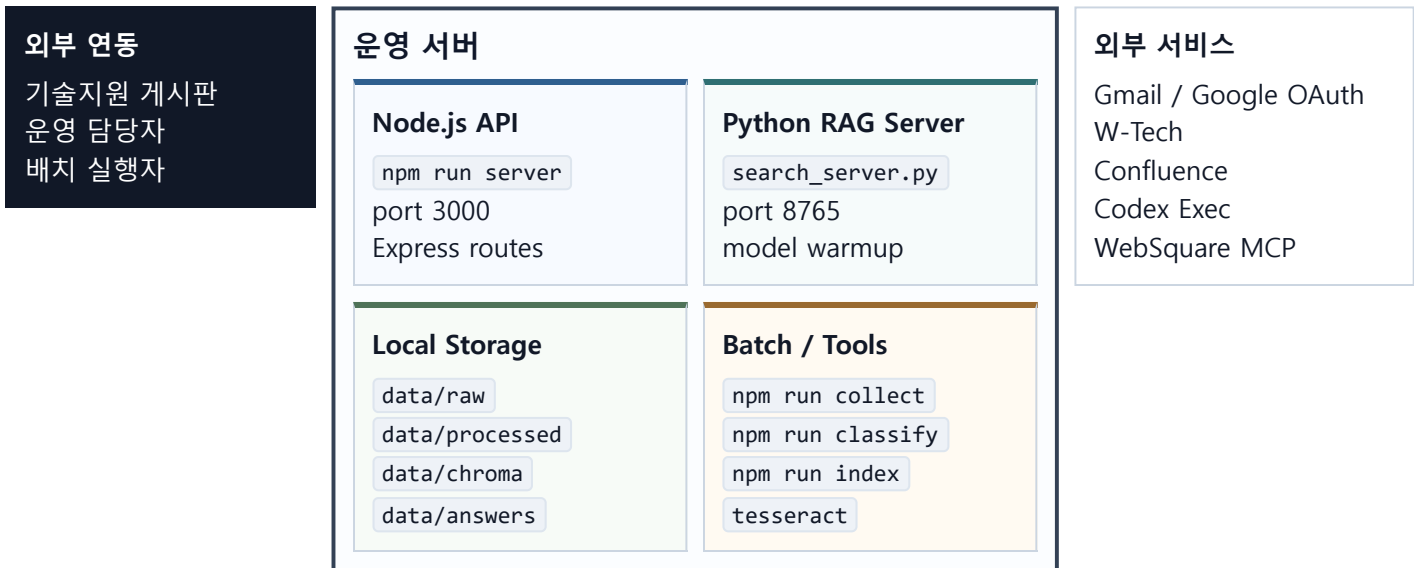
항목	설계	비고
임베딩 모델	<code>intfloat/multilingual-e5-base</code>	문서 <code>passage:</code> , 질의 <code>query:</code> prefix 사용
벡터 저장소	ChromaDB PersistentClient / Q&A 검색 컬렉션	<code>data/chroma</code> 에 저장
인덱싱	내용 기반 doc_id, batch 256, 증분 upsert	<code>--reset</code> 으로 전체 재색인 가능
검색 서버	<code>127.0.0.1:8765</code> HTTP 서버	모델과 BM25 인덱스 상주, 장애 시 subprocess 대체

성능 구조

질문마다 임베딩 모델을 새로 로딩하면 응답이 늦어집니다. 그래서 RAG 서버를 별도 프로세스로 띄워 모델과 BM25 인덱스를 메모리에 올려 둡니다. 첫 기동은 시간이 걸리지만, 운영 중 검색 응답 시간은 줄어듭니다.

5. 배포 및 운영 구조

그림 4. 단일 서버 배포 기준



운영 명령

구분	명령	역할
API 서버	<code>npm run server</code>	답변/검색 API 제공
수집	<code>npm run collect</code>	원천 지식 데이터 갱신
분류	<code>npm run classify</code>	통합 Q&A 카테고리 분류
인덱싱	<code>npm run index</code> / <code>npm run index:reset</code>	ChromaDB 검색 인덱스 갱신
답변 테스트	<code>node scripts/answer.js "질문"</code>	CLI 기반 파이프라인 검증

보안 및 장애 대응

인증/권한 <code>/api/answer</code> 는 API Key 인증을 적용합니다. 운영 설정은 <code>config/config.json</code> 으로 분리합니다.	민감정보 로그와 답변 저장 전에 <code>masking.js</code> 가 고객명, 이메일, 전화번호 등 민감 패턴을 제거합니다.	Fallback RAG 서버 장애 시 subprocess 검색으로 대체합니다. LLM 실패는 API 오류와 로그로 남깁니다.
--	--	---

장애 시나리오	처리 방식	확인 위치
RAG 서버 미응답	파이프라인이 Python subprocess 검색으로 자동 대체	API 서버 로그, RAG 서버 로그
Codex Exec timeout / quota	답변 API 실패 응답 반환	서버 stderr, 운영 감시 로그
OCR 실패	첨부 컨텍스트 없이 RAG/MCP 중심 답변 생성	파이프라인 attachment summary
미검증 API명 발견	1회 재생성 후 남은 항목은 경고로 표시	<code>apiVerifier.js</code> 결과